设计报告

钟志昊 3230105732

目录

一、 文档介绍 1
1.1 编写目的 1
1.2 文档范围 1
1.3 读者对象 1
1.4 术语与缩写解释 1
1.5 参考资料 2
二、 项目介绍 2
2.1 项目说明 2
2.2 项目背景 2
2.3 需求概述 3
三、 总体设计 5
3.1 技术介绍 5
3.2 系统结构 8
3.3 系统运行环境 12
四、 数据库设计 13
4.1 逻辑结构设计 13
4.2 物理结构设计 13
4.3 ER 图 15
五、 系统接口设计 15
5.1 统一说明 15
5.2 用户信息相关接口 16
5.3 图片类接口 20
5.4 内部方法 28
5.5 图片相关 28
5.6 JWT 相关 28
六、 界面原型 29
6.1 注册界面 29
6.2 登录界面 30
6.3 网站首页 30
6.4 上传图片界面 31
6.5 查看图片界面 31
6.6 用户管理界面 36

七、 系统出错设计 36
7.1 出错信息 36
7.2 防控措施 36
八、 附录 37
8.1 项目进度安排 37
8.2 备注 37

此处用于插入实验报告信息

一、文档介绍

1.1 编写目的

本软件产品功能设计说明书（SRS）旨在实现以下目标:

1. 明确定义软件系统的总体要求，为开发阶段的设计和编码工作提供坚实基础
2. 为软件系统的全面测试提供可靠的依据和标准
3. 为系统后续的维护和升级工作提供清晰、完整的参考文档
4. 为系统验收工作提供明确的判断标准，确保系统符合预期要求
5. 建立完善的需求变更管理体系，确保所有变更都有据可查

1.2 文档范围

本图片管理网站软件产品设计说明书全面描述了系统的功能特性及操作方式，详细阐述了系统的架构设计、接口设计、界面设计和数据库设计等核心内容，并提供了系统模块图、系统架构图，类图和 ER 图等可视化展示。

1.3 读者对象

本设计说明书面向以下读者群体:

1. 项目客户和利益相关者
2. 需求分析人员
3. 系统开发人员
4. 软件测试人员
5. 系统维护人员
6. 项目管理人员

1.4 术语与缩写解释

缩写及术语	解释
元数据	描述数据的内容、格式、来源和其他有关特性的数据信息

DTO	是一种常见的软件开发模式，用于在不同层之间传递数据，尤其在多层架构或跨进程调用中非常常用
类图	是面向对象建模（UML）中最常用、最核心的一种图，用来描述系统中的 类、属性、方法以及类之间的关系
ER 图	是一种在数据库设计中常用的图形化表示工具，用来描述系统中“数据实体”以及实体之间的“关系”
MCP	是一种为大模型设计的 标准化协议，用于让 AI 在统一、安全、结构化的方式下访问外部资源
MVC 模式	是一种常见的软件架构模式，主要用于实现 逻辑与界面的分离，让程序更清晰、更易维护、更方便扩展
部署图	展示系统的物理架构，描述软件模块部署在哪些硬件节点上及其之间的连接，适用于系统部署设计与分析

1.5 参考资料

序号	文档名称	作者	版本	发布日期
1	课程需求文档	授课老师	/	/

二、项目介绍

2.1 项目说明

本项目是浙江大学 2025-2026 秋冬学期《B/S 体系软件设计》课程的项目，旨在构建一个功能完善的图片管理网站。该系统支持用户注册登录后上传个人图片，并提供分类、检索、编辑、删除和查看等完整的图片管理功能。网站界面采用响应式设计，能够良好适配 PC 端和移动端设备。此外，系统还预留了 AI 图像识别和 AI 对话搜索等先进功能的扩展接口。项目要求提供完整的技术文档，包括设计报告、测试报告和用户手册等。

2.2 项目背景

随着智能手机、数码相机和社交媒体的快速发展，个人用户在日常生活中的数字图片和照片数量呈指数级增长。这些图片承载着用户珍贵的回忆、重要的工作资料或创作素材，具有重要的保存价值。然而，现有的图片管理解决方案普遍存在以下问题：

1. 数据分散与同步效率低下：图片文件分散存储在手机、电脑、云服务等多个平台，缺乏统一、集中的管理入口，导致查找困难且容易造成重复存储。
2. 手动分类与组织成本高昂：传统的管理方式（如文件夹分类）严重依赖用户手动操作。面对海量图片，手动整理分类不仅耗时巨大、容易出错，而且往往无法充分利用图片本身包含的丰富元数据信息。
3. AI 技术应用不足：目前市面上的图片管理网站普遍缺乏对 AI 技术的深度集成。随着 AI 技术的不断发展，已经可以实现使用 AI 进行图片分类和智能搜索，这将极大提升图片管理的效率和准确性。

2.3 需求概述

2.3.1 功能需求

2.3.1.1 需求概述

本项目是一个基于 B/S 架构的软件系统，根据需求文档的要求，需要实现以下核心功能：

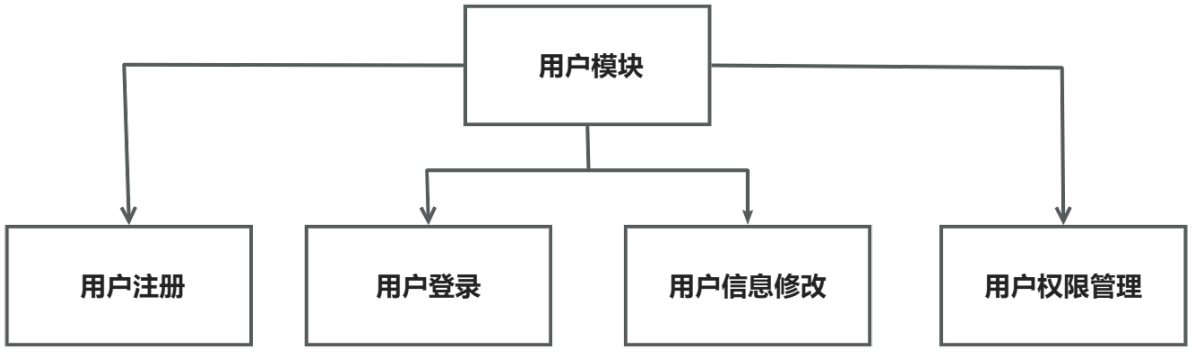
1. 实现用户注册、登录功能，用户注册时需要填写必要的信息并验证，如用户名、密码要求在 6 字节以上，email 的格式验证，并保证用户名和 email 在系统中唯一，用户登录后可以进行以下操作。
2. 通过 PC 或手机浏览器将照片或其他类型的图片上传到网站进行存储。
3. 能够通过照片的 exif 信息自动创建图片分类标签及其他辅助信息，如时间、地点、图片分辨率等。
4. 可以给图片增加自定义分类标签，方便检索。
5. 生产缩略图方便后续显示。
6. 图片信息保存在数据库中，方便后续查询。
7. 提供查询界面能根据各种条件查找图片。
8. 提供友好的展示界面，如选择一定的图片进行轮播显示等
9. 对选定的图片提供简单的编辑功能，如裁剪、修改色调等
10. 提供删除功能
11. 样式适配手机，开发手机 App 或能够在手机浏览器/微信等应用内置的浏览器中友好显示。

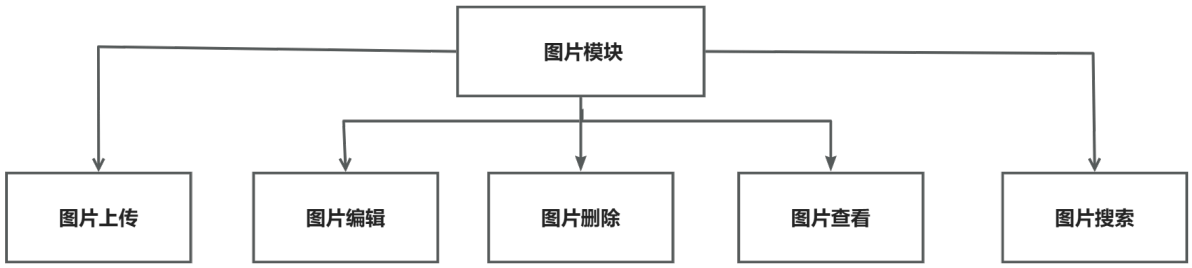
此外，系统还支持以下增强功能：

1. 调用 AI 模型分析图片，提供多类型的标签，如风景、人物、动物等
2. 提供 mcp 接口，能通过大模型对话方式检索网站上的图片。
3. 支持用户自定相册，方便用户对图片分类
4. 支持用户收藏图片，方便用户查找自己喜欢的图片

2.3.1.2 功能模块图

该系统主要面向两类用户：**普通用户**和**管理员**。普通用户只能查看和管理自己上传的图片，而管理员可以查看和管理所有用户的图片，以便进行系统监管。因此，项目的功能可以分为两大核心模块：用户管理模块和图片管理模块，功能模块图如下所示：





其中每个功能都有多种具体的实现方式，例如自动添加标签可以从图片的`exif`信息中自动提取，也可以由 AI 智能识别；图片查看可以是单张图片查看，也可以是图片轮播展示等。详细的技术实现方案将在后续章节中逐一介绍。

2.3.2 性能需求

系统应具备良好的整体性能表现，确保在实际使用过程中为用户提供流畅、稳定的体验。具体要求如下：

1. 响应速度：页面加载时间控制在 3 秒以内，用户操作响应时间不超过 5 秒，确保用户不会明显感觉到延迟。
2. 并发处理：具备一定的并发访问能力，支持多用户同时上传和查看图片。
3. 浏览器兼容性：兼容主流浏览器，包括但不限于 PC 端和移动端的 Firefox、Chrome、Safari、Edge 等，以及微信等应用的内置浏览器，且在不同浏览器及常见分辨率下均应保持界面一致性与功能正常。
4. 数据准确性：确保前端展示的数据能够实时同步后端数据库的最新信息，保证数据的一致性和准确性。
5. 性能可扩展性：根据实际访问量预留一定扩展空间，支持后续通过服务器扩展、负载均衡等手段提升并发能力。

2.3.3 安全性需求

考虑到用户上传的图片可能涉及个人隐私，系统必须具备完善的安全防护机制。具体要求如下：

1. 密码安全：
   - 密码在数据库中存储加盐哈希值，防止密码泄露
   - 要求用户设置 8 位以上的复杂密码（包含大小写字母、数字及特殊字符），以提高密码强度，降低暴力破解的风险
   - 设置密码错误尝试次数限制
   - 系统定期提示用户更新密码，以防遗忘及提升账户安全性。
2. 攻击防护：防范 SQL 注入等常见网络攻击
3. 会话管理：对于长时间无操作的登录会话，系统应自动强制登出，防止账户被盗用，但同时在用户有操作时会话不能过期，以保证用户的良好体验。
4. 输入验证：对前端用户输入进行严格验证，拦截非法字符和恶意代码，比如集成 AI 搜图后需要防止提示词注入。
5. 传输安全：对于重要或敏感的信息，比如用户图片，需要加密传输，可以选择使用 HTTPS，或在前后端之间约定如何加密。
6. 秘钥安全：本系统中会涉及一些第三方 API 的秘钥，这些秘钥不应该被传输到前端或者远端仓库，相关 API 调用应该完全在后端完成。

2.3.4 可维护性需求

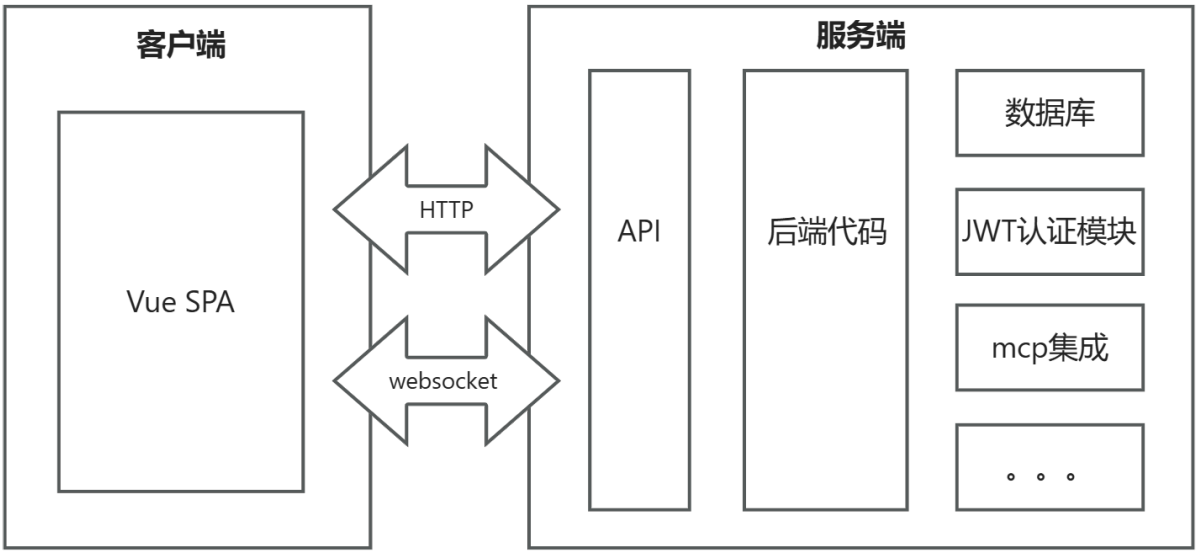
系统应具备良好的可维护性，确保长期稳定运行和持续优化。具体要求如下：

1. 模块化设计：采用清晰的模块化架构，项目结构层次分明，充分考虑模块内部结构的紧密性与模块间的独立性，确保各模块功能明确、职责单一，内部实现细节对外部屏蔽，模块之间通过清晰、标准化的接口进行通信。
2. 文档完备：提供完整、清晰的技术文档，结合图表进行详细说明，需要有需求文档（需求已由授课老师给出，包含在设计文档中），设计文档，测试文档，使用说明书和本地开发指南等。
3. 代码规范：严格遵循代码规范，前端使用 EsLint 控制 TS/JS 代码规范，后端使用相应的格式化工具严格规范格式，并且代码需要提供充足的注释。
4. 测试体系：建立完善的测试体系，支持单元测试、集成测试等多种测试方式，后端使用单元测试框架给出完备的单元测试，前端主要通过集成测试区保证功能完整性。
5. 版本管理：使用 Git 进行代码版本管理，确保开发、测试和发布流程的规范有序，遵循 Conventional Commits 规范，在 `dev` 分支进行开发，最终 `merge` 到主分支。
6. CICD：使用 `github actions` 进行流水线配置，在编译、测试和静态检查提交代码，并生成标准化测试报告，通过全部检查后将代码打包成 docker image 推送到 dockerhub，便于后续部署。

三、总体设计

3.1 技术介绍

本项目采用前后端分离的 Web 开发架构（BS 架构），后端提供 RESTful API 供前端调用，同时实现了 MCP（Model Context Protocol）服务器，将前端作为 MCP 客户端，实现对 AI 智能查找图片功能的支持。系统处理流程如下图所示：



整个项目的技术选型如下：

- 前端：Vue3(TypeScript) + Element Plus 组件库 + Vue Router + npm 包管理器 + Vite 构建工具
- 后端：Gin 框架(Go) + GORM
- 数据库：MySQL + Redis
- MCP：MCP TypeScript SDK + MCP Python SDK

3.1.1 前端

3.1.1.1 TS

本项目前端语言选择了 TypeScript(TS)，它作为 ES6+的一个强类型超集，允许开发者在编码阶段明确定义变量、函数参数、返回值及对象属性的数据类型，从而在编译阶段即可捕获潜在的类型错误，有效避免因类型不匹配导致的运行时异常，显著提升了代码的可靠性与可维护性。与此同时，由于 TypeScript 最终会被编译成纯 JavaScript 执行，因此它与现有的 JavaScript 生态系统实现了无缝兼容，开发时无需担心无法使用某些 JS 库的问题，同时还能通过社区维护的类型定义文件（如 DefinitelyTyped）获得出色的类型支持，确保了项目的平滑迁移和长期可扩展性，真正实现了开发效率与代码质量的双重提升。

3.1.1.2 Vue 3

本项目前端框架选用 Vue 3，这是一款遵循渐进式理念构建的现代化前端框架，其核心设计赋予开发者从简单到复杂项目的灵活伸缩能力，支持根据需求平滑地集成路由、状态管理等周边技术。Vue 3 的核心优势体现在其高效的声明式渲染机制、细粒度响应式系统和强大灵活的组件模型等方面。本项目将全面采用组合式 API（Composition API）进行开发，将特定功能相关的数据、计算属性、方法以及生命周期逻辑组织在一起，形成高内聚、低耦合的复合函数，使得项目的代码结构更加清晰、健壮且易于长期迭代。

3.1.1.3 Element Plus

本项目组件库选用 Element Plus，该组件库作为对 Vue 3 生态的深度适配和全面升级，由饿了么前端团队官方维护，拥有坚实的生产实践背书和活跃的社区支持，是当前 Vue 3 技术栈下公认最流行、最成熟的企业级 UI 框架之一。它面向设计师与开发者，提供了一套完整且设计语言统一的组件系统，涵盖了从数据展示（、数据录入、导航、反馈到布局等几乎所有后台管理系统所需的界面元素，真正做到开箱即用，能极大提升开发效率。

3.1.2 后端

3.1.2.1 Go

本项目后端开发语言选用 Go（Golang），该语言由 Google 设计并开源，自诞生之初便以系统级编程为定位，在语法设计上贯彻“少即是多”的哲学，摒弃复杂的特性与继承体系，采用极其简洁直观的语法结构，配合极其快速的编译速度，显著提升了开发迭代效率和代码的可读性。此外，Go 语言提供了现代化且高效的开发工具链，包括内置的`go mod`包依赖管理、开箱即用的单元测试框架、强大的代码格式化与静态分析工具。

3.1.2.2 Gin

本项目后端框架选用 Gin，这是一个基于 Go 语言构建的轻量级、高性能 Web 框架，充分继承并发挥了 Go 语言在高并发场景下的原生优势。框架提供了强大而精炼的核心功能集，包括灵活可扩展的中间件支持机制，允许开发者通过插件方式便捷地集成认证、日志、限流等通用功能；其路由引擎不仅性能高效，更支持灵活的参数匹配和便捷的路由分组，能够清晰优雅地组织 API 结构；同时，Gin 内置了高效的 JSON 绑定与验证功能，可智能地将请求数据自动反序列化为结构体并执行数据校验，大大简化了接口开发流程；框架还提供了完善的错误恢复机制，能够自动捕获并处理运行时异常，确保服务不会因未处理的崩溃而中断。这些精心设计的特性使 Gin 成为构建高吞吐量 RESTful API 服务、微服务以及快速开发高可维护性后端应用的理想选择，在保障开发效率的同时提供企业级应用的稳定性和性能表现。

3.1.3 数据库

3.1.3.1 MySQL

本项目关系型数据库管理系统选用 MySQL。MySQL 作为全球最流行、应用最广泛的关系型数据库管理系统之一，最初由瑞典 MySQL AB 公司开发，目前由 Oracle 公司负责运营和维护。它以其久经考验的稳定性、卓越的性能和丰富的功能特性而著称，完全能够满足本项目对数据一致性、事务安全和系统高可用的严格要求。

3.1.3.2 ORM 框架

为高效地与 MySQL 进行交互，本项目将使用 `GORM` 作为对象关系映射（ORM）框架。`GORM` 是基于 Go 语言开发的全功能 ORM 库，它提供了一套高度抽象、语义清晰且对开发者极为友好的 API 接口，能够将复杂的手动 SQL 编写工作转化为直观的 Go 方法链调用，从而极大地简化了对数据库的增删改查、事务处理、关联查询等各类操作。同时，GORM 在底层实现了严格的类型安全，确保 Go 语言中的结构体与数据库表结构之间的映射准确且可靠。

3.1.3.3 Redis

本系统还将引入 Redis 作为 MySQL 的高性能缓存层与内存数据存储。Redis 是一个开源、基于内存的键值对存储系统，其数据操作主要在内存中完成，因此具备极高的读写速度与极低的访问延迟，能够轻松应对海量并发请求。在本项目中，Redis 将主要承担数据缓存的职责，将 MySQL 中频繁访问的热点数据暂存于内存，显著减轻数据库的直接压力，提升系统整体响应速度。

为达成上述目标，本项目将采用稳定高效的 `go-redis` 客户端库与 Redis 服务器进行交互，该库提供了类型安全的 API 和与 Go 生态高度契合的调用方式，能够无缝集成到 Gin 框架中。

3.1.4 认证与鉴权

本系统的认证与鉴权采用 JWT（JSON Web Token）技术，这是一种遵循 RFC 7519 开放标准的紧凑且自包含的安全令牌方案。JWT 由头部、载荷和签名三部分组成，通过数字签名确保信息在传输过程中不被篡改，其设计紧凑性使其可通过 URL 参数、HTTP 请求头等多种方式在客户端与服务端之间高效传输。JWT 的核心优势在于其无状态特性，服务端无需在内存或数据库中存储会话信息，仅需通过验证令牌的签名即可确认其有效性，这极大简化了系统架构并显著提升了横向扩展能力；同时，其标准化的结构使其天然具备卓越的多平台适用性，能够无缝支持 Web、移动端及微服务等各种应用场景。

在本项目中，我们将使用稳定可靠的 `github.com/golang-jwt/jwt/v4` 库来实现完整的 JWT 生命周期管理，包括使用高强度密钥进行令牌签发、精确解析令牌中的声明信息以及严格验证令牌签名与有效期，该库提供了类型安全的 API 和灵活的配置选项，能够有效防范常见的安全风险，确保用户身份验证与权限分发机制既安全可靠又高效运行。

3.1.5 逆地理编码 API

考虑到用户交互友好性与数据呈现直观性的核心需求，本项目集成了由高德地图提供的业界领先的逆地理编码（Reverse Geocoding）API 服务。该服务能够接收经纬度坐标点作为输入，通过其背后强大的地理信息数据库和精准的解析算法，迅速且准确地反向查询出该坐标点所对应的、结构化的标准中文地址描述。其返回的结果不仅精确到省、市、区/县三级行政区域，更能进一步提供街道、门牌号等更为细致的地址信息，从而将抽象的数字坐标转化为用户能轻松理解的真实世界地址，为基于地理位置的查询服务提供了有力支持。

3.1.6 图像识别 API

本系统的 AI 图像识别功能将基于阿里云视觉智能开放平台提供的图像分类 API 服务进行构建。该服务依托阿里云在大数据和深度学习领域的深厚技术积累，采用在业界权威基准测试中达

到 92.7% 平均精度的先进模型，能够对上传的 JPEG、PNG、BMP 等常见格式图像进行高效精准的多维度内容理解。其核心能力不仅限于基础的对象识别，更涵盖了场景解析、元素检测、颜色分析、风格判断乃至特定领域如菜品、商品等专项识别，可自动生成一个涵盖物体、场景、概念、属性等多维度的精确标签集合。

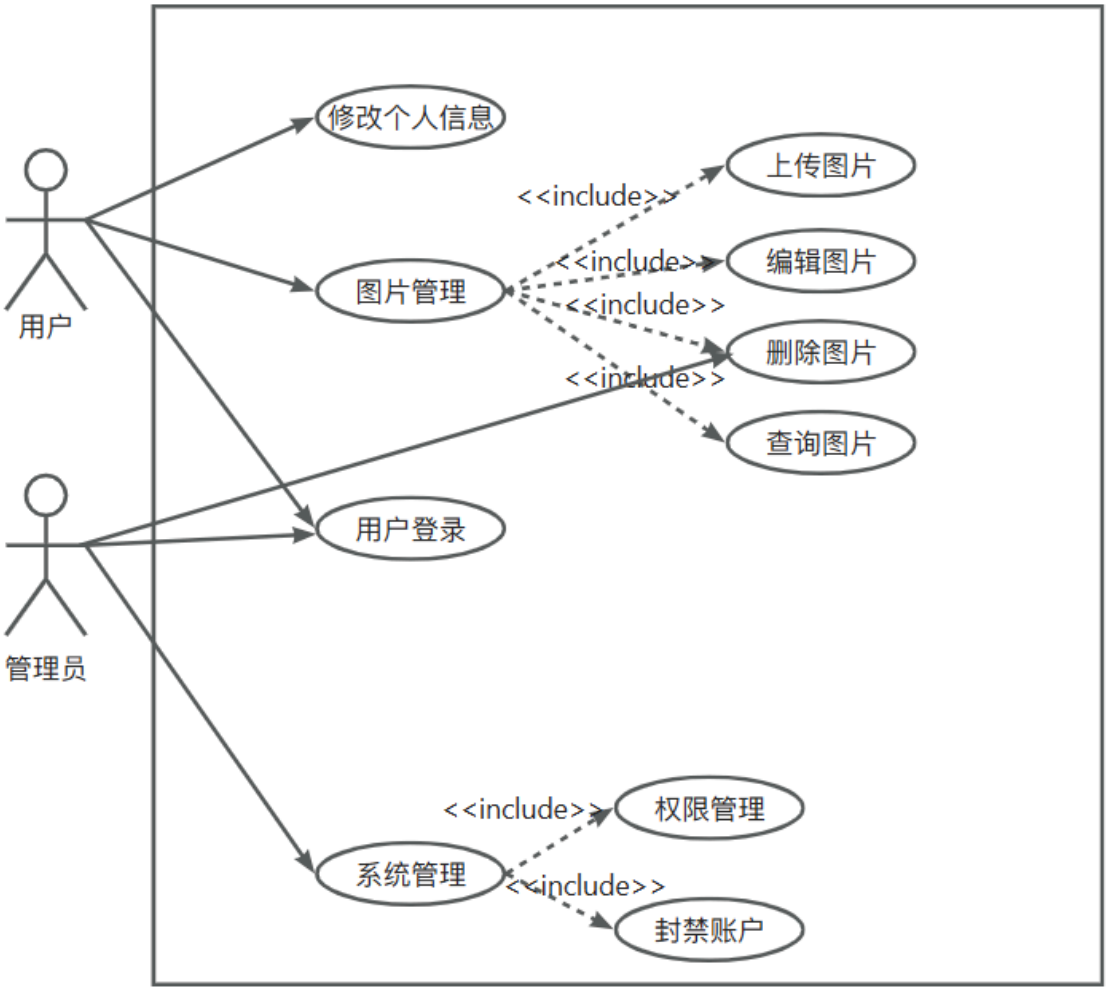
3.1.7 MCP

MCP（Model Context Protocol，模型上下文协议）是一种开放协议，旨在实现大型语言模型（LLM）应用与外部数据源、工具和服务之间的无缝集成。本项目将在前端使用 TypeScript 实现 MCP 客户端，然后使用 Python 实现 MCP 服务器，使系统能够支持用户通过对话方式查询网站图片。TypeScript 和 Python 均使用官方的 MCP SDK。

3.2 系统结构

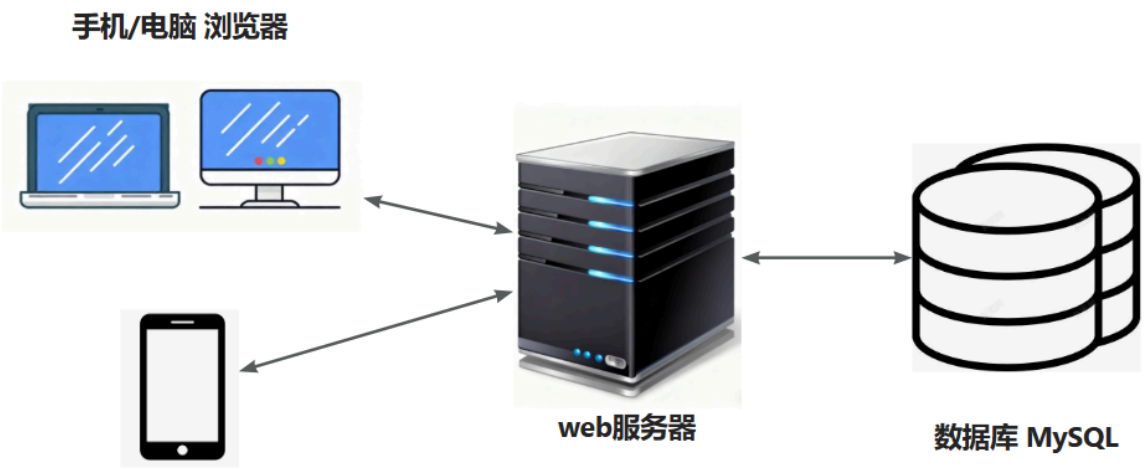
3.2.1 用例图

根据需求分析，得出的总用例图如下所示:



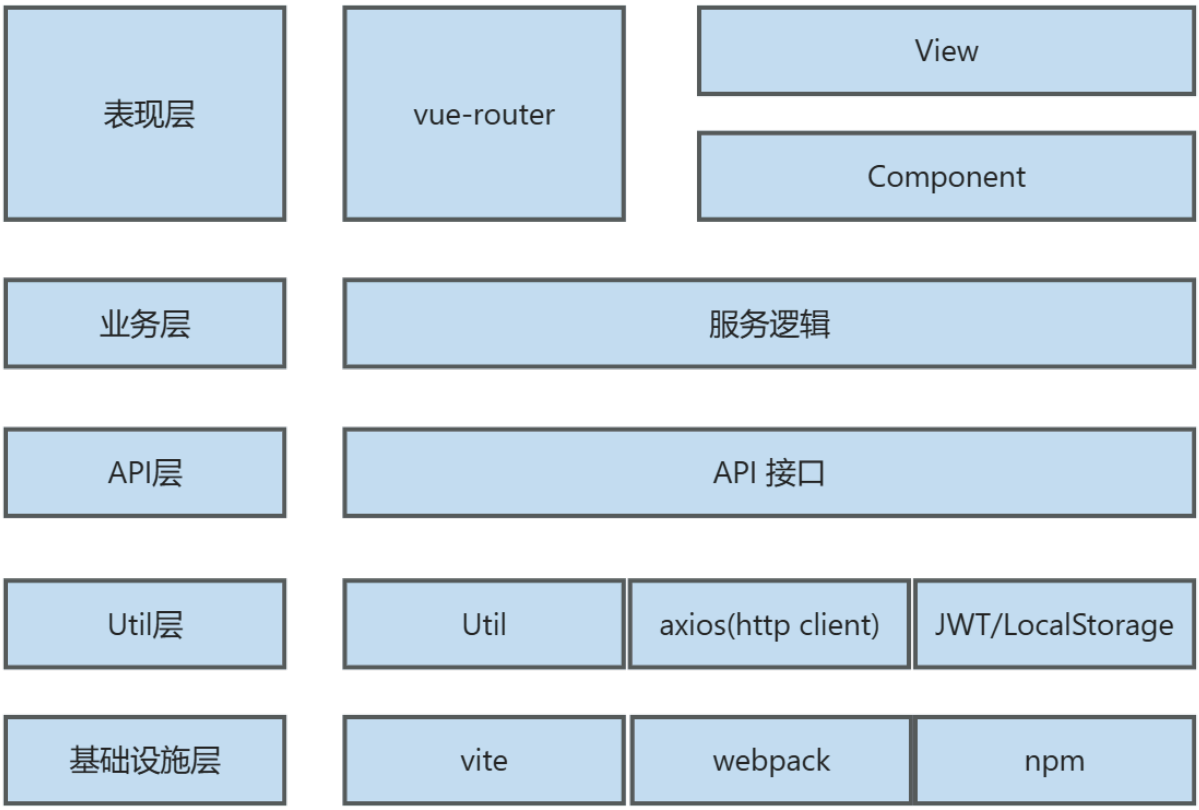
3.2.2 系统总体架构图

系统的总体架构如下图所示，其中客户端支持 PC 和手机等多种设备访问，后端服务器可以接收前端网页发出的 HTTP 请求并进行处理，将需要持久化的数据存储在关系型数据库中。（此处只展示了核心逻辑，像用于优化性能的 Redis 被省略了）



3.2.3 前端架构图

该项目前端主要采用 Vue 3 作为核心开发框架，在 UI 组件方面，项目选用了 Element Plus，在构建与依赖管理方面，项目使用 `npm` 作为包管理器，用于管理第三方依赖与版本控制。项目在开发阶段采用 `Vite` 作为构建工具，同时，在生产环境中使用 `webpack` 进行打包。

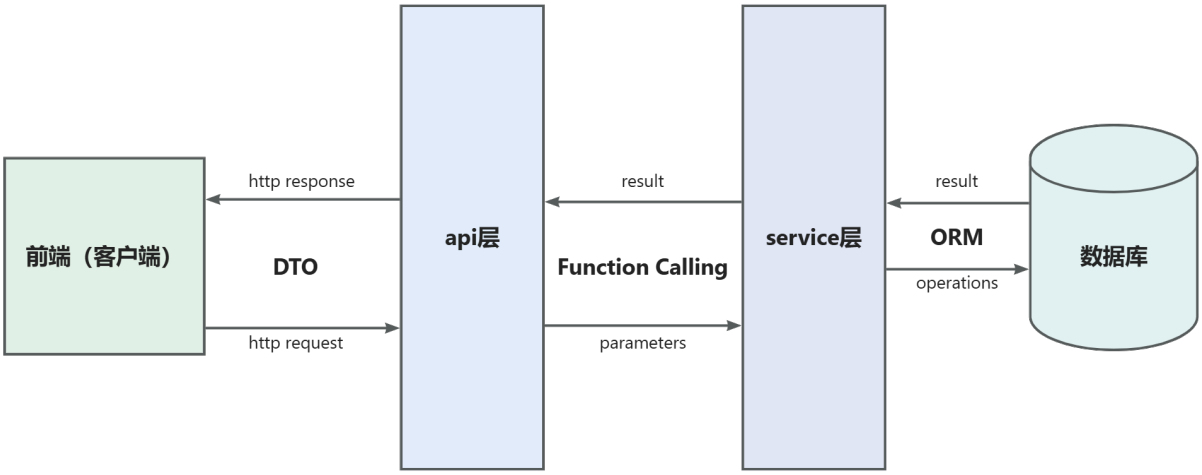


3.2.4 后端架构图

后端遵循经典的 MVC（Model–View–Controller）分层架构设计思想，将系统逻辑清晰拆分为模型层、业务控制层与展示层，以提升系统的可维护性、扩展性和代码复用性。其中：

- **Model（模型层）** 负责与数据库交互，结合 ORM 框架完成数据的增删改查操作，并对业务实体进行封装，使数据处理逻辑更加结构化、稳定和可复用。

- **Controller（控制层）**作为前端或客户端请求进入系统的入口，负责接收 HTTP 请求、调用对应的业务服务方法、处理数据转换（如 DTO）、并将处理结果按照约定格式返回给前端。
- **Service（业务逻辑层）**承担核心业务处理逻辑，负责组合调用、规则校验、流程分发等，使控制层只专注请求响应，模型层只专注数据管理，从而实现关注点分离。



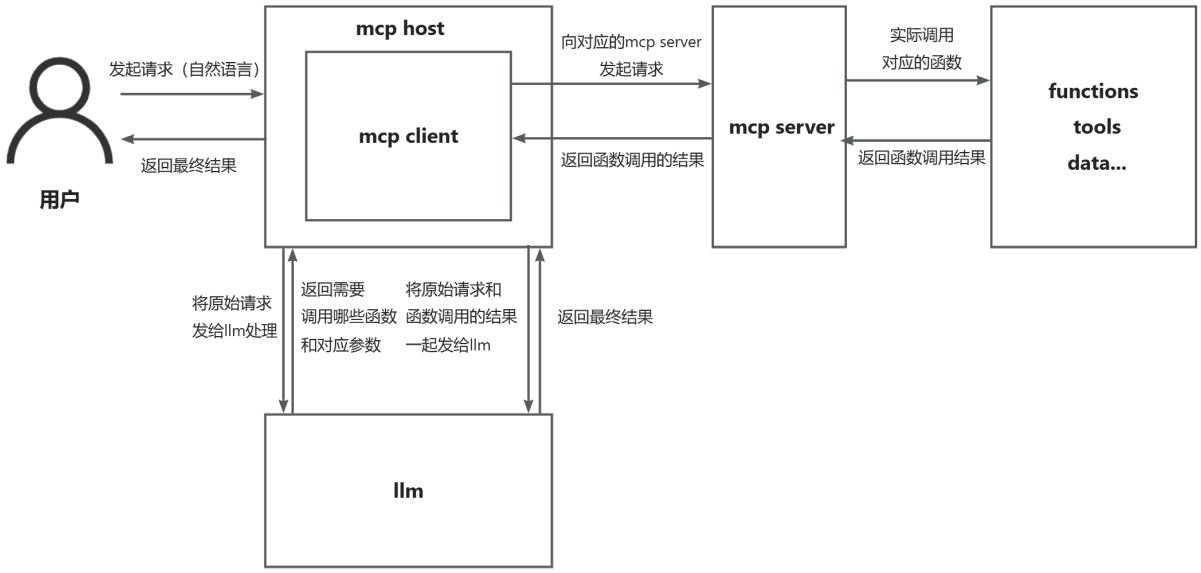
3.2.5 MCP 架构图

MCP（Model Context Protocol）的交互流程，使用户请求能够通过大模型与外部业务能力（如函数、工具或数据接口）进行协同处理。整体流程如下：

1. 用户发起请求：用户通过自然语言输入指令，该指令首先被系统接收并交由 MCP Host 进行解析和调度。
2. MCP Host 调用 LLM 进行意图分析：MCP Host 将用户原始请求发送给大语言模型（LLM），由 LLM 负责分析用户需求，判断该任务是否需要调用外部工具或函数。如果需要，LLM 会返回：
   - 需要调用的函数或 API 名称
   - 对应的调用参数

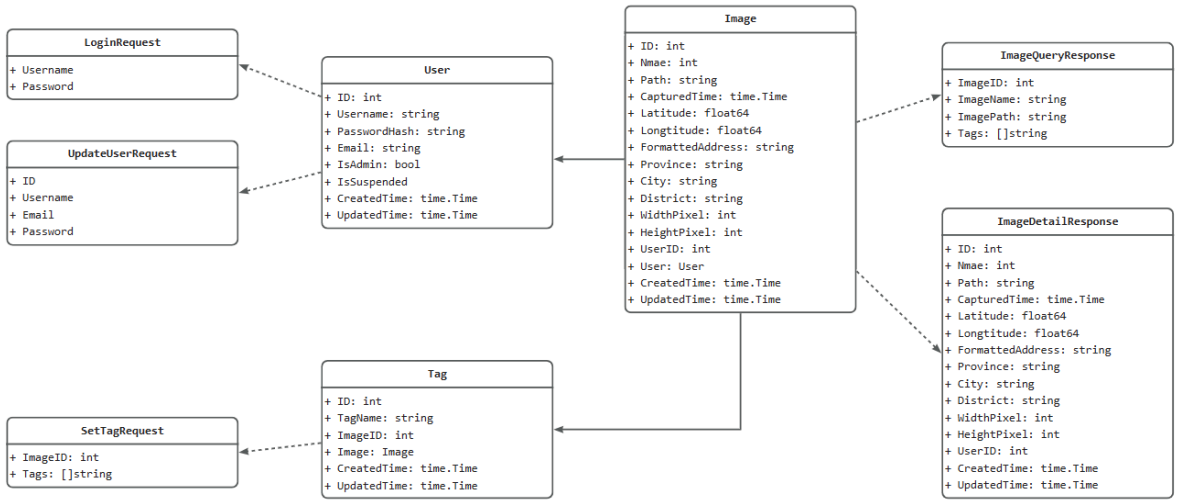
   MCP Host 接收该结构化指令后继续下一步。
3. MCP Host 调用 MCP Client 发出功能请求：MCP Host 将 LLM 推断出的函数调用交由 MCP Client 实际执行。MCP Client 会向对应的 MCP Server 发送请求，并携带参数与调用意图。
4. MCP Server 完成实际业务逻辑执行：MCP Server 接收到请求后，负责执行具体任务，如：
   - 调用业务函数
   - 执行工具脚本
   - 查询业务系统或数据库

   执行完成后将结果返回给 MCP Client。
5. 将调用结果反馈回 LLM：MCP Client 将执行结果交还给 MCP Host，再由 MCP Host 将该结果与原始上下文一起返回给 LLM。LLM 继续生成自然语言解释、最终答案或决定是否继续下一步调用。
6. 最终结果返回用户：LLM 生成最终用户可读的响应内容，并通过 MCP Host 返回，使用户看到最终处理结果。

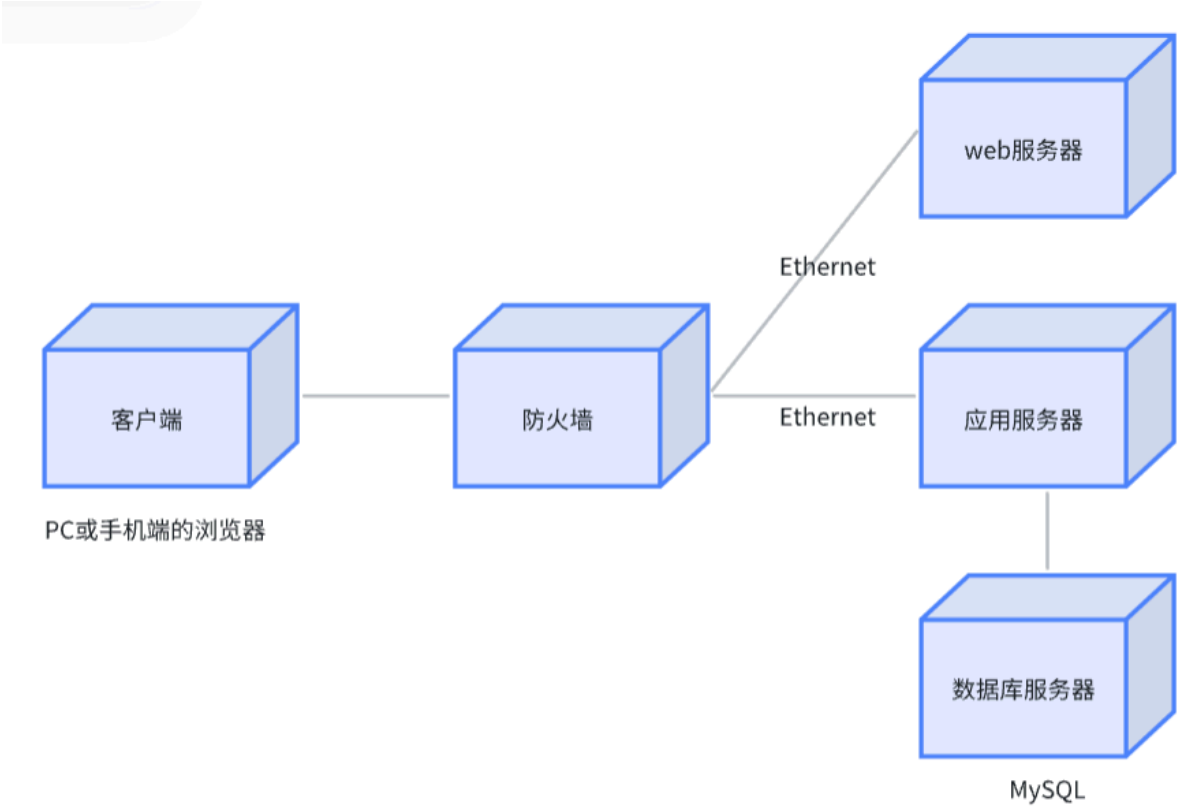


3.2.6 类图

虽然本项目后端使用的 Go 是非面向对象的语言，不过在实际当中很多地方仍会遵循 `OOP` 的思想，故还是绘制了类图。



3.2.7 部署图



3.3 系统运行环境

系统运行环境包括硬件环境和软件环境，以下按照客户端和服务器端分别列举运行环境要求：

3.3.1 客户端

对用户端的硬件要求如下：

名称	要求
CPU	以 Windows 为例 9th COREi5 以上
内存	2GB 以上
硬盘	256GB 以上
显示器	全彩显示
键盘，鼠标	可正常使用

对客户端的软件要求如下：

- 操作系统：Windows 7 以上、Linux 或 macOS
- 浏览器：主流浏览器，如 Edge、Firefox 和 Chrome 等

3.3.2 服务器端

对服务器端的硬件要求如下：

名称	要求
CPU	以 Windows 为例 11thCOREi5 以上

名称	要求
内存	4GB 以上
硬盘	512GB 以上

对服务器端的软件要求如下：

类型	名称	版本
操作系统	Windows 10/11、Linux 或 macOS	
关系型数据库	MySQL	9.0 以上
key-value 数据库	Redis	8.0 以上
后端环境	Go	1.24 以上
前端环境	npm	10.0 以上

上述软件依赖均通过 Docker 部署，最终项目中会提供 Docker Compose 文件，所以服务端需要有较新版本（v25.0 以上）的 Docker。

四、数据库设计

4.1 逻辑结构设计

本系统的数据库表设计如下三张，具体说明见下方表格：

1. User(id, name, password_hash, email, is_admin, is_suspended, created_time, updated_time)
2. Image(id, name, time, longitude, latitude, province, city, district, formatted_address, width_pixel, height_pixel, user_id, created_time, updated_time)
3. Tag(id, name, image_id, created_time, updated_time)

4.2 物理结构设计

4.2.1 User

User 表用于存储用户的基本信息，主要包含用户名、账号和密码等信息，保证 ID、用户名和邮箱唯一。此外，密码存储加盐哈希后的值。

字段名	类型	描述	备注
id	int	用户 id	主键、非空、自增
name	varchar(64)	用户名	非空，唯一
password_hash	char(60)	密码哈希	非空
email	varchar(64)	邮箱	非空，唯一
is_admin	bool	是否为管理员	非空，默认 false
is_suspended	bool	是否被封禁	非空，默认 false
created_time	datetime	创建时间	GORM 自动设置
updated_time	datetime	更新时间	GORM 自动设置

4.2.2 Image

Image 表用于存储图片的元信息。由于图片文件体积较大，不适合直接存储在数据库中，因此本系统采用文件系统存储图片文件，数据库中仅存储图片的文件名。通过图片存储目录和文件名即可定位到对应的图片文件。

字段名	类型	描述	备注
id	int	图片 id	主键、非空、自增
name	varchar(128)	图片名	非空，唯一
time	datetime	拍摄时间	可空
longitude	decimal(10, 6)	经度	可空
latitude	decimal(10, 6)	纬度	可空
province	varchar(16)	省份	可空
city	varchar(16)	市	可空
district	varchar(16)	区	可空
formatted_address	varchar(64)	完整位置	可空
width_pixel	int	宽像素	可空
height_pixel	int	高像素	可空
user_id	int	用户 id	非空，外键
created_time	datetime	创建时间	GORM 自动设置
updated_time	datetime	更新时间	GORM 自动设置

4.2.3 Tag

Tag 表用于存储图片的标签信息，实现图片的分类管理功能。

字段名	类型	描述	备注
id	int	标签 id	主键、非空、自增
name	varchar(16)	标签名	非空，唯一
image_id	int	图片 id	非空，外键
created_time	datetime	创建时间	GORM 自动设置
updated_time	datetime	更新时间	GORM 自动设置

4.2.4 缩略图存储

本项目中图片存储在后端服务器，在 Gin 框架中配置静态文件目录，以便前端直接根据 URL 显示图片。本项目将每位用户的图片放在各自的文件夹（名称为用户 ID）下存储，原图和缩略图分别存储在不同的文件夹中。后端大致的目录结构如下所示：

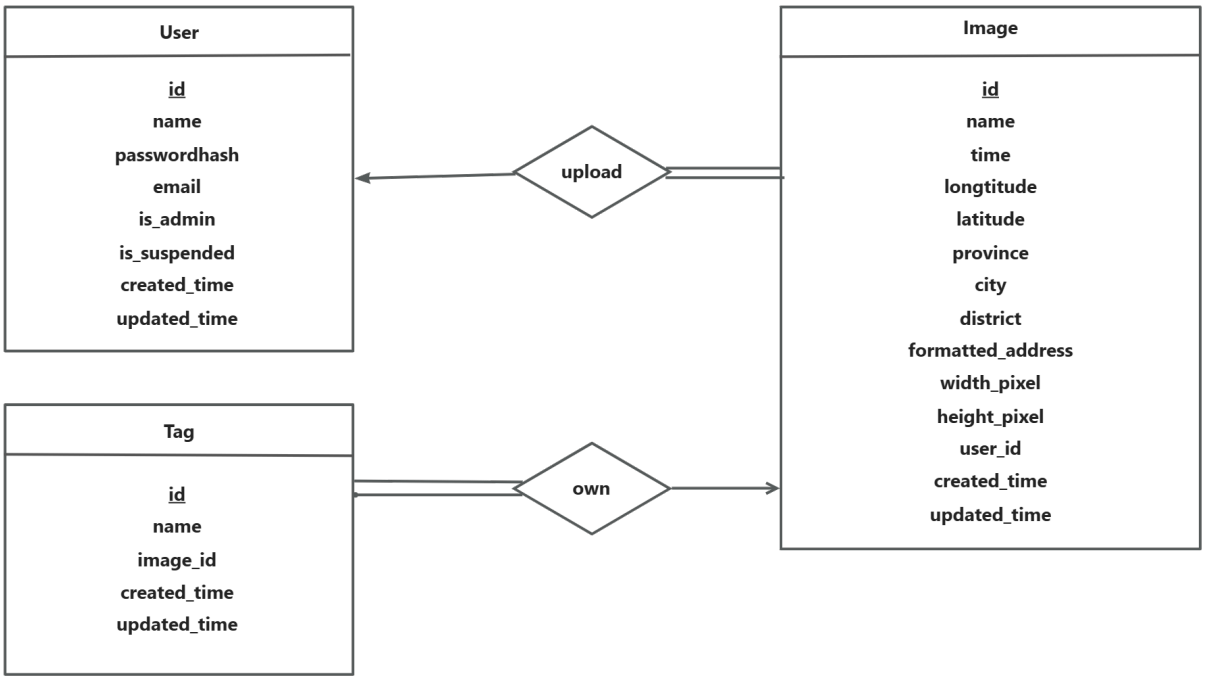
```
server/ # 后端目录
|______ public/ # 静态文件目录
|       |______ user_id # 具体的用户ID
|               |______ origin # 原图目录
|               |______ thumbnail # 缩略图目录
|______ # 其它目录
```

由于原图和缩略图的路径都可以通过数据库中的图片元信息计算得出，因此不需要额外在数据库中存储缩略图的路径。此外，由于用户可能上传同名图片，为了避免文件覆盖，本项目会为每张图片的名称加上一个 UUID（Universally Unique Identifier）。

4.3 ER 图

User、Image 和 Tag 均为强实体，它们之间存在如下约束：

- 一位用户可以有多张图片，一张图片必须被某一位用户所有
- 一张图片可以有多个标签，一个标签必须归属于某些图片



五、系统接口设计

5.1 统一说明

> 🔥 **提示**
>
> 由于目前系统仍处于开发阶段，API 接口将在编码过程中进行调整，最终的 API 接口与本文档可能存在差异。

系统接口主要用于前后端交互，本项目前端使用 axios 作为 HTTP 客户端，与后端进行异步交互，而后端则提供一系列 API 和 Service，并绑定到相应路由上。本系统的 API 设计一定程度上遵循 RESTful 规范，具体设计如下。此外，还有一些接口用于后端内部调用，比如提取图片的 `exif` 信息，这个方法仅在每次上传图片时由后端调用。

出入参主要是针对用于前后端进行交互的接口，对于入参，本项目在参数较少时选择使用 HTTP 的 query 参数，在参数较多时选择在 body 中使用 JSON 格式传输 DTO（数据传输对象）。

对于出参，本项目规定统一的返回格式如下所示：

```
{
  "code": 200, // HTTP返回码
```

```
  "message": "", // 返回的消息
  "data": [] // 返回的数据，可以是任意类型
}
```

其中 code 是 HTTP 返回码，与实际返回码保持一致；message 是后端返回的提示信息，这两部分的作用比较固定；data 是后端返回的具体数据，以下接口返回参数说明中只解释 data 部分。

5.2 用户信息相关接口

用户信息类接口主要使用 User 表的数据，并对 User 表进行增删改查操作，提供的 URL 均以 user 开头，具体的 API 主要有：

5.2.1 用户注册

接口 URL	/user/register
HTTP 请求类型	POST
请求参数类型	application/json
请求参数示例	`{ "username": "李华", "email": "3230106666@zju.edu.cn", "password": "123456" }`
请求参数说明	username、email 和 password 分别是用户名、邮箱和密码
返回值示例	`{ "code": 0, "message": "string", "data": { "id": 0, "username": "string", "email": "string", "isAdmin": true, "createTime": "string", "updateTime": "string" } }`
返回值说明	id 是用户 ID，username 是用户名，email 是用户邮箱，isAdmin 表示用户是否是管理员，createdTime 和 updatedTime 分别是用户的创建和修改时间
接口简介	用于用户注册，会在数据库表中添加对应信息，默认不是管理员

5.2.2 用户登录

接口 URL	/user/login
HTTP 请求类型	POST
请求参数类型	application/json
请求参数示例	`{ "username": "李华",`

	<pre>"password": "123456" }</pre>
请求参数说明	username 和 password 分别是用户名和密码

返回值示例	<pre>{
	"code": 0,
	"message": "string",
	"data": {
	"token": "string",
	"user": {
	"id": 0,
	"username": "string",
	"email": "string",
	"isAdmin": true,
	"createTime": "string",
	"updateTime": "string"
	}
	}
	}</pre>

返回值说明	token 是生成的 JWT，user 中是用户的各项信息
接口简介	用于用户登录，验证用户输入的密码是否正确，若正确则返回 JWT 和用户信息，这两个东西将存储在浏览器 localStorage 中以便后续使用

5.2.3 根据 ID 查询用户

接口 URL	/user/getUserById
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	id:integer
请求参数说明	id 是用户 id

返回值示例	<pre>{
	"code": 0,
	"message": "string",
	"data": {
	"id": 0,
	"username": "string",
	"email": "string",
	"isAdmin": true,
	"createTime": "string",
	"updateTime": "string"
	}
	}</pre>

返回值说明	用户的各项信息
接口简介	用于根据 ID 获取用户信息，仅限管理员使用

5.2.4 查询所有用户

接口 URL	/user/getAllUsers
HTTP 请求类型	GET

请求参数类型	query
请求参数示例	page:integer pasize:integer
请求参数说明	分页查询的页数和每页数量

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"token": "string",
	"user": {
	"id": 0,
	"username": "string",
	"email": "string",
	"isAdmin": true,
	"createTime": "string",
	"updateTime": "string"
	}
	}
	}

返回值说明	新的 token 和更新后的用户信息
接口简介	用于用户更新自己的个人信息

5.2.5 更新用户信息

接口 URL	/user/updateUser
HTTP 请求类型	PUT
请求参数类型	application/json

请求参数示例	{
	"id": 2,
	"username": "王华",
	"email": "222333@qq.com",
	"password": "abcdef"
	}

请求参数说明	id、username、email 和 password 分别是用户 id、新用户名、新邮箱和新密码

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"token": "string",
	"user": {
	"id": 0,
	"username": "string",
	"email": "string",
	"isAdmin": true,
	"createTime": "string",
	"updateTime": "string"
	}
	}
	}

返回值说明	token 是生成的 JWT，user 中是用户的各项信息
接口简介	用于用户登录，验证用户输入的密码是否正确，若正确则返回 JWT 和用户信息，这两个东西将存储在浏览器 localStorage 中以便后续使用

5.2.6 找回密码

接口 URL	/user/retrievePassword
HTTP 请求类型	POST
请求参数类型	application/json

请求参数示例	{
	"new_pasword": "11223344",
	"id": 0,
	"username": "李华",
	"email":"3230106666@zju.edu.cn"
	}

请求参数说明	new_password 是新的密码，id、username 和 eamil 是对应账户的 id、用户名和邮箱

返回值示例	{
	"code": 0,
	"message": "string",
	"data": null
	}

返回值说明	根据返回码判断是否成功
接口简介	用于用户找回自己的密码

5.2.7 授予管理员

接口 URL	/user/grantAdmin
HTTP 请求类型	PUT
请求参数类型	query
请求参数示例	id:integer
请求参数说明	id 是用户 id

返回值示例	{
	"code": 0,
	"message": "string",
	"data": null
	}

返回值说明	根据返回码判断是否成功
接口简介	用于授予管理员权限，仅限管理员使用

5.2.8 撤销管理员

接口 URL	/user/revokeAdmin
HTTP 请求类型	PUT
请求参数类型	query

请求参数示例	id:integer
请求参数说明	id 是用户 id
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据返回码判断是否成功
接口简介	用于撤销管理员权限，仅限管理员使用

5.2.9 封禁用户

接口 URL	/user/suspend
HTTP 请求类型	POST
请求参数类型	query
请求参数示例	id:integer duration:integer
请求参数说明	id 是用户 id，duration 是持续时间（按分钟计算）
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据返回码判断是否成功
接口简介	用于封禁一个用户一定时长，仅限管理员使用

5.2.10 解封用户

接口 URL	/user/unsuspend
HTTP 请求类型	POST
请求参数类型	query
请求参数示例	id:integer
请求参数说明	id 是待解封用户 id
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据返回码判断是否成功
接口简介	用于提前解封一个被封禁了的账户，仅限管理员使用

5.3 图片类接口

图片类接口主要使用 Image 和 Tag 表的数据，并对这两张表进行增删改查操作，提供的 URL 均以 image 开头，具体的 API 主要有：

5.3.1 上传图片

接口 URL	/image/uploadImage
HTTP 请求类型	POST
请求参数类型	multipart/form-data
请求参数示例	image:string<binary> imageName:string
请求参数说明	image 是图片的二进制数据, imageName 是自定义图片名称

返回值示例	{
	"code": 0,
	"message": "string",
	"data": null
	}

返回值说明	根据状态码判断是否成功
接口简介	用于上传图片，后端会将图片通过文件系统保存到对应目录，图片 EXIF 信息和其他元信息会被保存到数据库中

5.3.2 设置标签

接口 URL	/image/setTags
HTTP 请求类型	PUT
请求参数类型	application/json

请求参数示例	{
	"imageID": 0,
	"tags": []
	}

请求参数说明	imageID 是图片 ID，tags 是想要添加的标签（字符串数组）

返回值示例	{
	"code": 0,
	"message": "string",
	"data": null
	}

返回值说明	根据状态码判断是否成功
接口简介	用于为指定图片设置标签，采用直接覆盖方式，所以增删标签均使用这个接口

5.3.3 更新图片

接口 URL	/image/updateImage
HTTP 请求类型	POST
请求参数类型	multipart/form-data
请求参数示例	image:string<binary> imageID:integer
请求参数说明	image 是新的图片的二进制数据，imageID 是图片 ID
返回值示例	

	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据状态码判断是否成功
接口简介	用于更新图片，主要用于用户在前端编辑图片后的更新操作

5.3.4 删除图片

接口 URL	/image/deleteImage
HTTP 请求类型	DELETE
请求参数类型	query
请求参数示例	imageID:integer userID:integer
请求参数说明	imageID 是图片 ID，userID 是图片拥有者的用户 ID
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据状态码判断是否成功
接口简介	用于删除图片

5.3.5 查询所有图片

接口 URL	/image/getAllImages
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer
请求参数说明	分页查询的页数和每页数量

返回值示例	<pre>{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}</pre>

返回值说明	总页数+图片核心信息
接口简介	用于分页查询所有图片，仅限管理员使用

5.3.6 查询个人所有图片

接口 URL	/image/getImageByID
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer
请求参数说明	分页查询的页数和每页数量

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于分页查询用户自己的所有图片

5.3.7 根据标签查询图片

接口 URL	/image/getImageByTag
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer tag:string
请求参数说明	分页查询的页数和每页数量，tag 是查询的标签

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}

	`],`
	`"pageTotal": 0`
	`}`
	`}`

返回值说明	总页数+图片核心信息
接口简介	用于分页根据 tag 查询用户自己的所有图片

5.3.8 根据时间区间查询图片

接口 URL	/image/getImageByTimeRange
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer startTime:string endTime:string
请求参数说明	分页查询的页数和每页数量，以及时间区间的始末

返回值示例	`{`
	`"code": 0,`
	`"message": "string",`
	`"data": {`
	`"images": [`
	`{`
	`"image_id": 0,`
	`"image_name": "string",`
	`"image_path": "string",`
	`"tags": []`
	`}`
	`],`
	`"pageTotal": 0`
	`}`
	`}`

返回值说明	总页数+图片核心信息
接口简介	用于分页根据时间区间查询用户自己的所有图片

5.3.9 根据省份获取图片

接口 URL	/image/getImageByProvince
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer province:string
请求参数说明	分页查询的页数和每页数量

返回值示例	`{`
	`"code": 0,`
	`"message": "string",`

	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于根据省份分页查询用户自己的所有图片，直辖市算作省

5.3.10 根据城市查询图片

接口 URL	/image/getImageByCity
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer district:string
请求参数说明	分页查询的页数和每页数量

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于根据城市分页查询用户自己的所有图片

5.3.11 根据区县查询图片

接口 URL	/image/getImageByDistrict
HTTP 请求类型	GET
请求参数类型	query

请求参数示例	page:integer pageSize:integer district:string
请求参数说明	分页查询的页数和每页数量

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于根据区县分页查询用户自己的所有图片

5.3.12 查询图片详情

接口 URL	/image/getImageDetail
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	imageID:integer
请求参数说明	图片 ID

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"id": 0,
	"name": "string",
	"path": "string",
	"captured_time": "string",
	"latitude": 0,
	"longitude": 0,
	"formatted_address": "string",
	"province": "string",
	"city": "string",
	"district": "string",
	"width_pixel": 0,
	"height_pixel": 0,
	"user_id": 0,
	"user": {
	"id": 0,
	"username": "string",
	"email": "string",
	"isAdmin": true,

	<pre>"createTime": "string", "updateTime": "string" }, "created_time": "string", "updated_time": "string" } }</pre>
返回值说明	图片的详细信息，包括标签和 EXIF 信息等
接口简介	用于根据图片 ID 查询图片的详细信息

5.3.13 获取图片文件

接口 URL	/image/getImageFile
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	imageID:integer
请求参数说明	图片 ID
返回值示例	string<binary>
返回值说明	真正的图片文件
接口简介	用于获取图片文件，主要是下载图片时需要

5.3.14 通过大模型检索图片

接口 URL	/image/getImageThroughLLM
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	prompt:string
请求参数说明	检索图片的提示词

返回值示例	<pre>{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	LLM 检索到的图片
接口简介	用于与大模型交互，通过对话的方式检索图片

5.4 内部方法

在本系统中，有些功能是仅在后端使用，不涉及返回信息给前端。

5.5 图片相关

5.5.1 图片 `exif` 信息提取

接口名称	GetExif
参数	图片文件
返回值	图片 `exif` 信息
接口简介	根据图片文件解析其中的 `exif` 信息
注意事项	注意图片可能因为各种原因没有 `exif` 信息

5.5.2 位置信息解析

接口名称	ReverseGeocode
参数	精度和纬度
返回值	具体的地理信息，比如浙江省杭州市西湖区浙大路 38 号浙江大学
接口简介	封装对高德地图逆地理编码 API 的调用，根据经纬度给出具体地理信息
注意事项	注意只有国内的地址可解析，国外地址解析结果为空

5.5.3 AI 识图生成标签

接口名称	GenerateTags
参数	图片文件
返回值	图片标签数组
接口简介	调用阿里云视觉智能开放平台提供的图像分类 API 服务，识别图片并生成图片标签
注意事项	仅保留可信度高于 75% 的标签

5.6 JWT 相关

5.6.1 JWT 生成

接口名称	CreateToken
参数	用户 ID，用户名，邮箱，是否为管理员
返回值	JWT token
接口简介	根据用户信息生成新的 JWT 令牌
注意事项	无

接口名称	ParseToken
参数	JWT 令牌

返回值	令牌是否有效
接口简介	检验 JWT 令牌是否有效
注意事项	无

接口名称	RefreshToken
参数	JWT 令牌
返回值	新的 JWT 令牌
接口简介	检验旧的 JWT 令牌是否有效，然后生成新的 JWT 令牌并将旧的令牌作废
注意事项	JWT 令牌本身是没有任何办法可以使其提前过期的，但可以通过黑名单的方式屏蔽掉被认为无效的令牌

六、界面原型

> **提示**
>
> 本设计报告完成于项目完全实现之前，下面的界面设计仅供参考。

以下是项目主要内容的界面原型，主要展示核心交互逻辑和想要达成的效果，部分界面可能还比较简单，后续会进行优化。

6.1 注册界面

在这个界面用户可以填写自己的基本信息，包括用户名、邮箱和密码，然后注册自己的账户。如果注册成功（用户名和邮箱都不重复，并且密码符合要求），系统会自动跳转到登录页面。



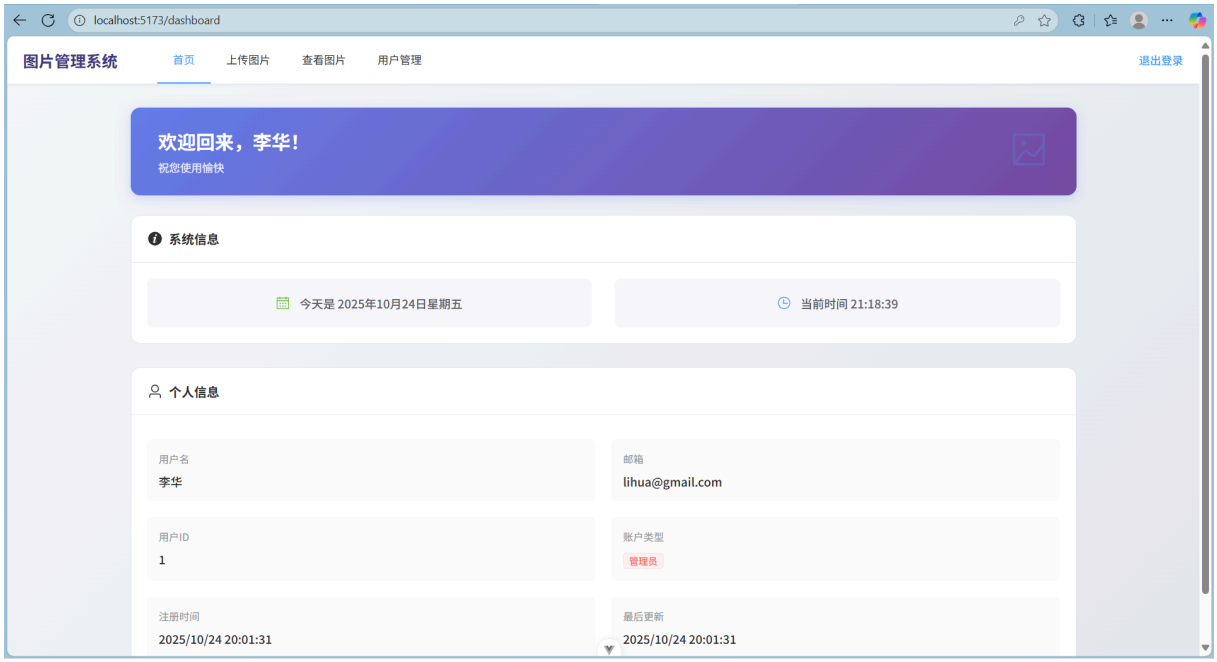
6.2 登录界面

在这个界面用户可以输入自己的用户名和密码，然后点击登录按钮。如果登录成功（密码正确，且账号未被封禁），则跳转到网站首页。如果其他页面产生了 HTTP 状态码为 401 Unauthorized 的请求（说明未登录或者账号被封禁），则会被重定向到登录页面。



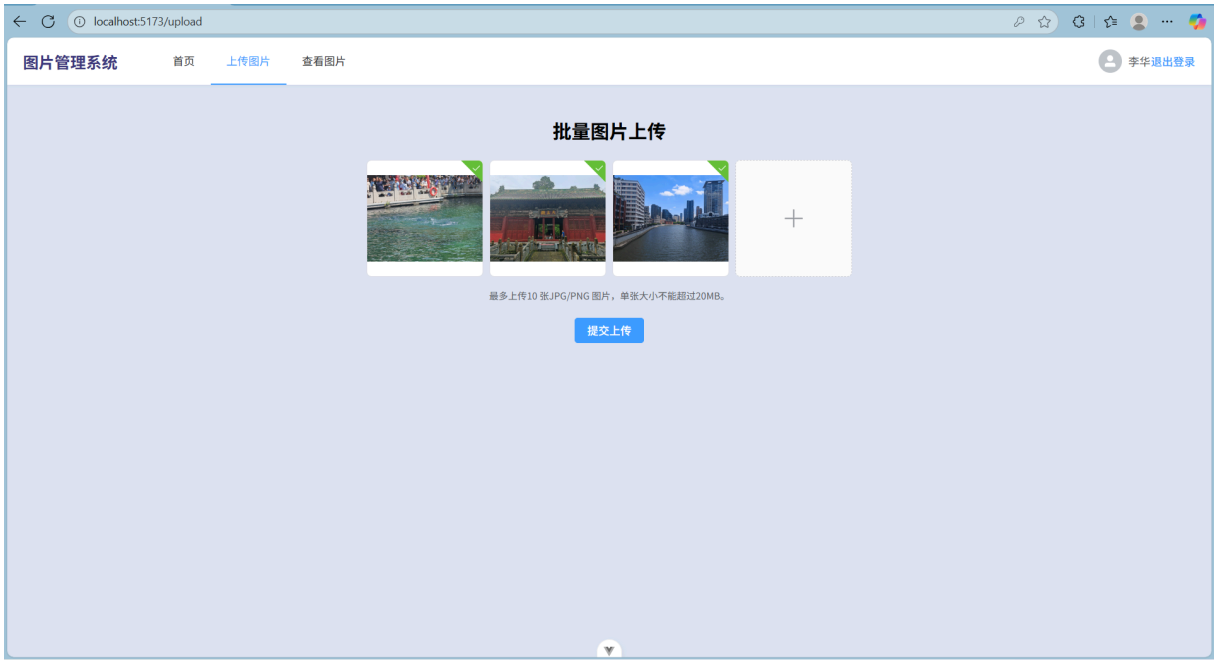
6.3 网站首页

在这个界面会显示一些系统信息和用户的个人信息，可以点击上边栏跳转到其他页面。上边栏中的“用户管理”只会在管理员的前端渲染，并且在通过上边栏跳转时也会进行权限验证。



6.4 上传图片界面

这个页面用于上传图片，支持批量上传（一次最多 10 张）。点击加号方框即可从本地挑选图片上传，已经上传到前端的图片会在前端显示预览图，点击提交上传按钮才会正式将图片发送到后端存储。



6.5 查看图片界面

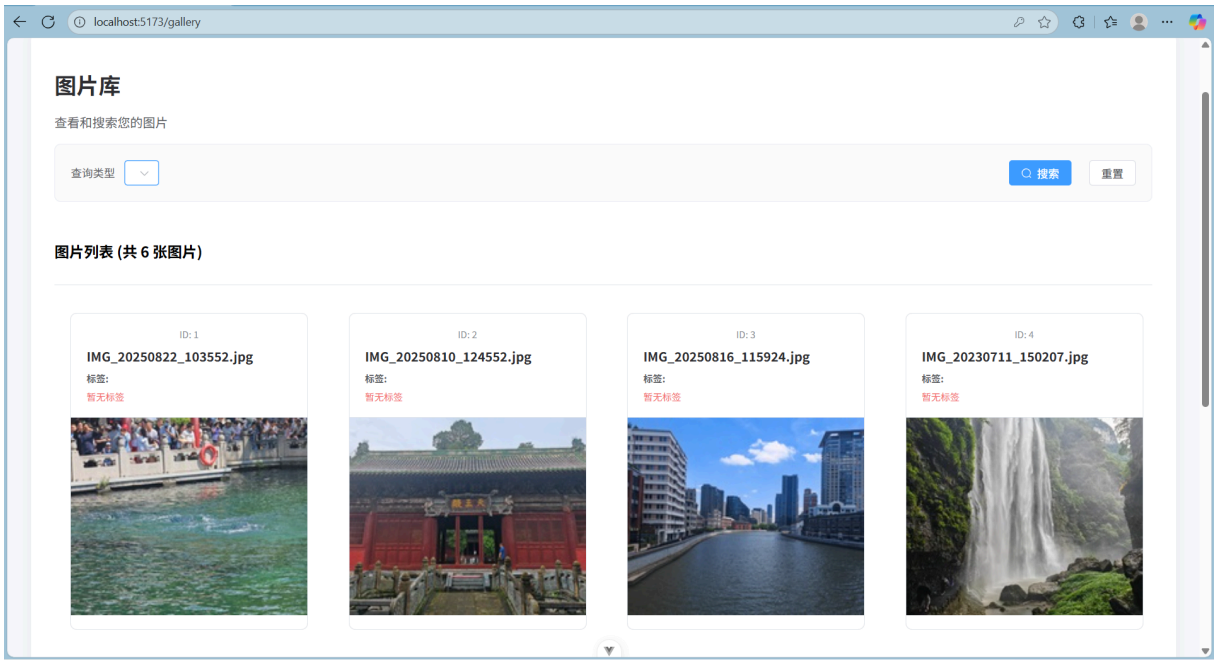
这个界面的功能比较丰富，下面分别介绍各个功能：

可以在这个界面按照不同条件查询图片，支持的查询条件包括：

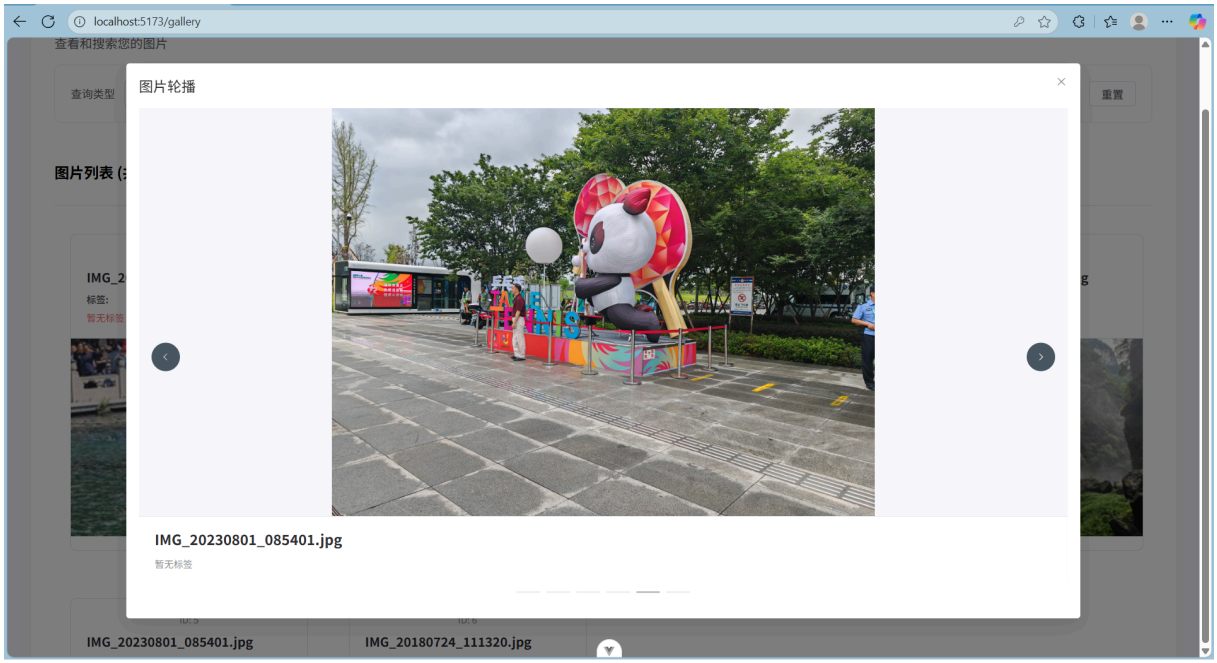
- 个人所有图片
- 图片标签
- 拍摄时间区间
- 省市区地理位置



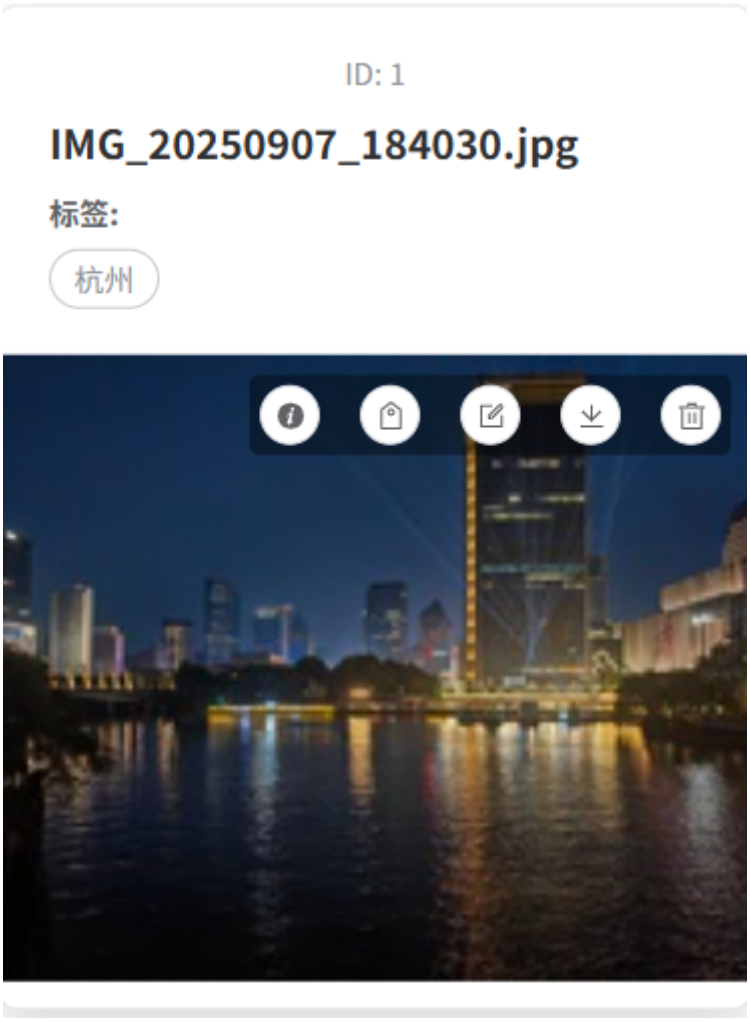
查询结果如下图所示，图片会依次排列并显示最核心的信息，即图片 ID、图片名和标签。



点击图片可以进行图片轮播，并且会自动切换到被点击的图片。



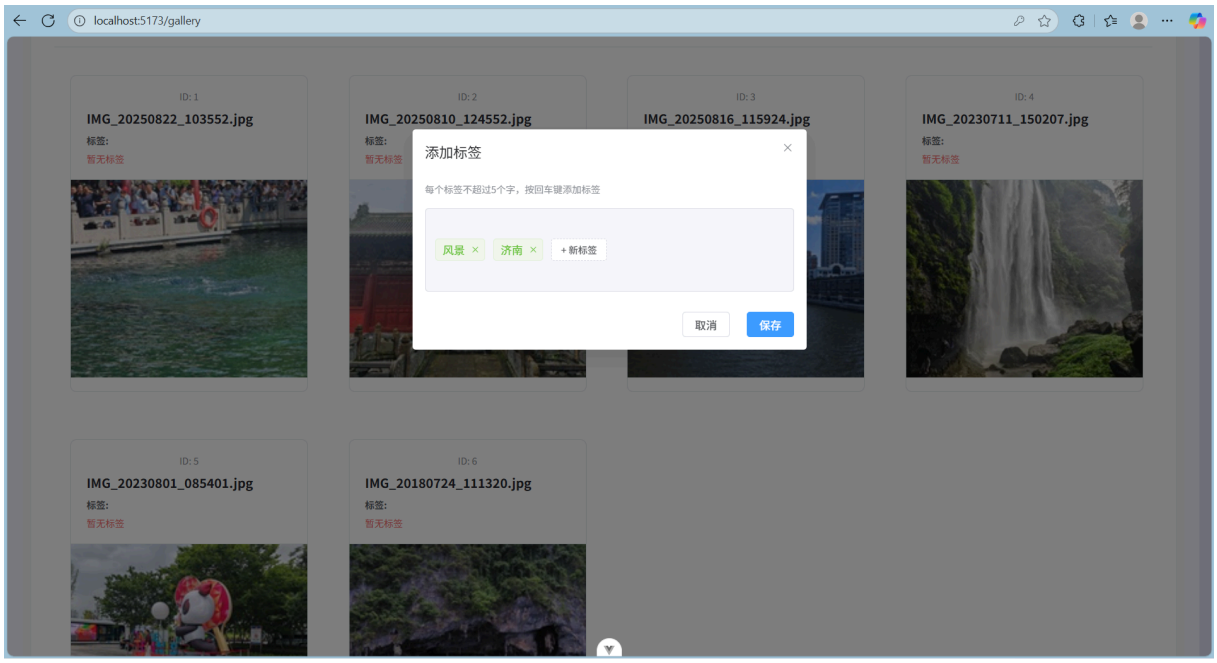
当鼠标悬停在图片上时，图片右上角会浮现四个图标，分别对应查看图片详情、添加标签、编辑图片、下载图片和删除图片。



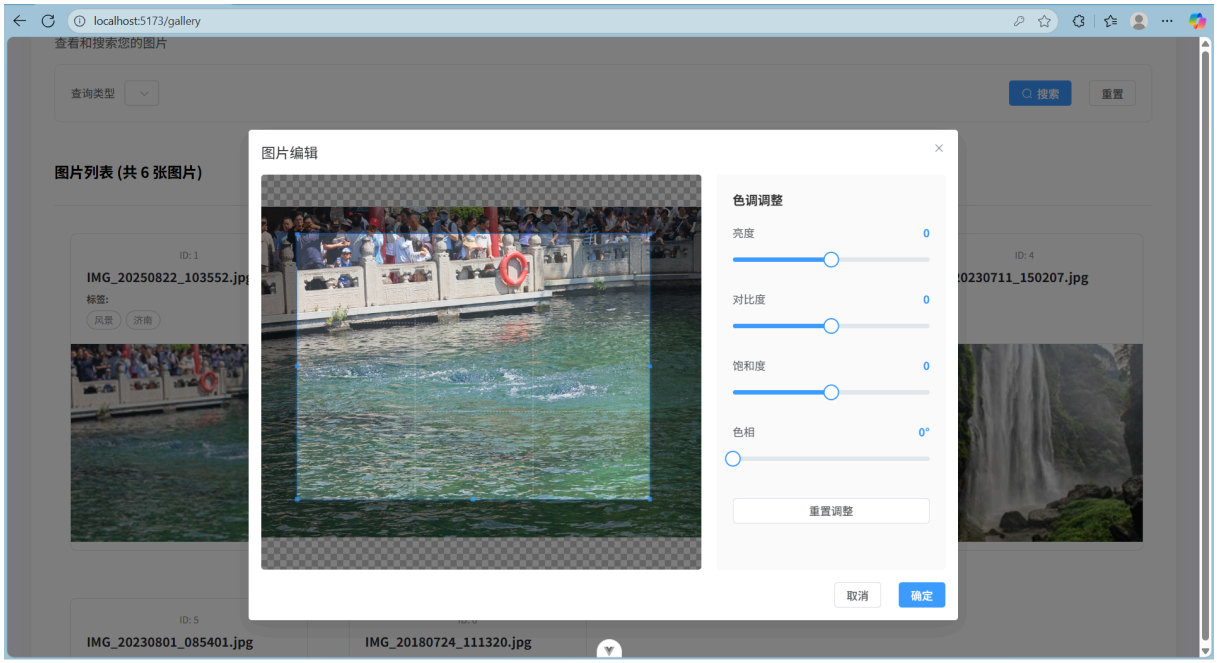
点击查看图片详情图标后，会弹出对话框显示图片原图和详细信息（主要是图片的 EXIF 信息）。



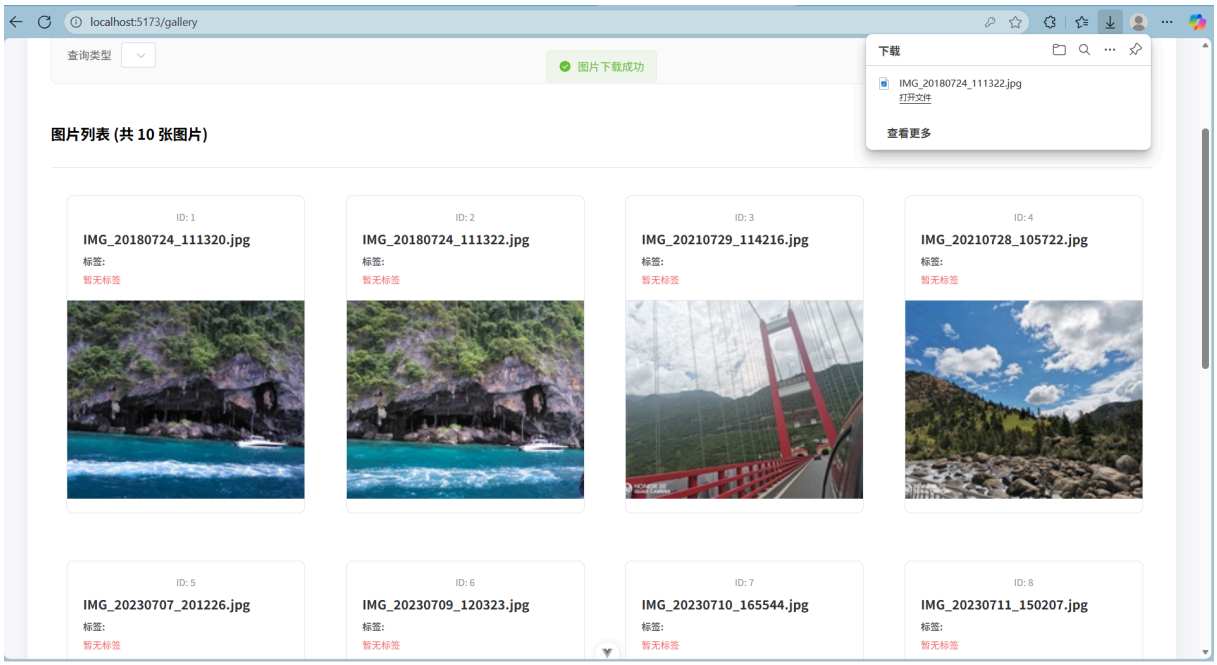
点击添加标签图标后，会弹出对话框显示当前图片的标签，可以添加新的标签，也可以删除已有的标签。



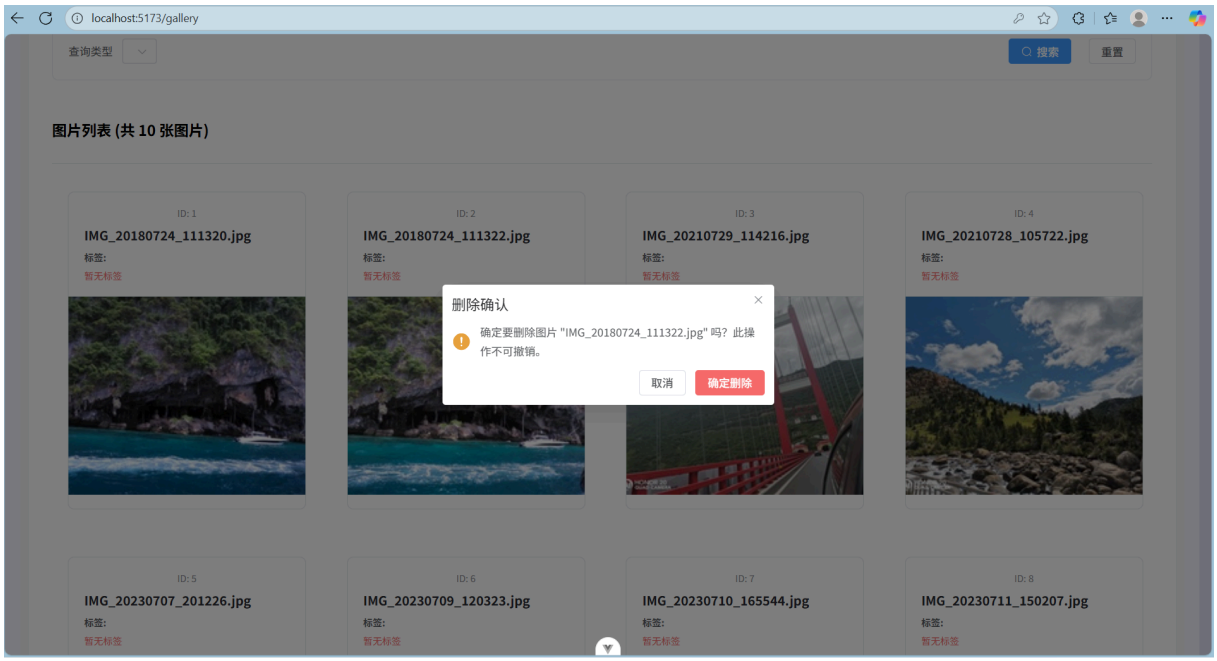
点击编辑图片图标后，会弹出对话框显示当前图片原图，并且可以裁剪图片，调整图片的亮度、对比度、饱和度和色相等属性。



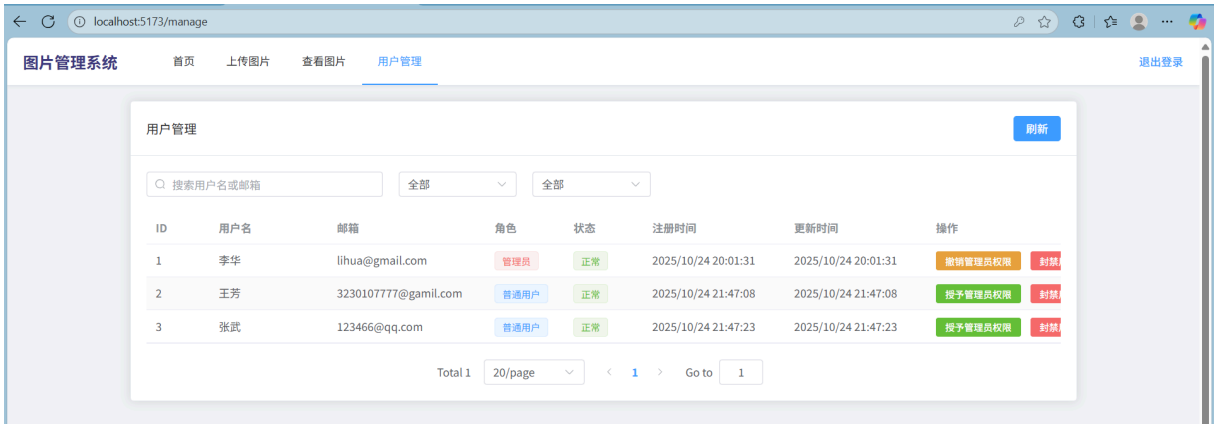
点击下载图片图标后，图片原始文件会被发送到前端，然后自动在浏览器中下载。



点击删除图片图标后，如果确认删除，则图片文件和元信息将分别从文件系统和数据库中删除。



6.6 用户管理界面



七、系统出错设计

7.1 出错信息

系统通过统一的错误码和友好的信息反馈异常情况，关键错误方式及处理方式如下：

错误类型	错误信息提示	可能原因	处理方法
后端连接失败	弹窗提示“系统繁忙，请稍后再试”	1. 后端启动失败 2. ip 配置错误	1. 检查配置文件 2. 重启服务
权限不足	页面提示“您无权限执行此操作”	普通用户视图越权访问管理员功能	记录日志并跳转至默认页面
未登录直接访问业务页面	弹窗提示“您尚未登录”	直接通过 URL 访问业务页面	自动返回登录页面
已在业务页面操作但 JWT 无效	弹窗提示“您已退出登录”	长时间未操作导致 JWT 过期	自动返回登录页面
访问不存在的页面	弹窗提示“您访问的页面不存在”	URL 输入了错误地址	跳转到 404 页面
文件格式错误	提示“图片格式仅支持 png 和 jpg”	上传其它格式的图片或信息缺损的图片	强制重新选择图片

7.2 防控措施

本项目主要通过系统日志来监控项目状态：

1. 在日志中记录所有 HTTP 请求的信息，包含发起操作者 IP、时间戳及详细参数
2. 日志存储在后端文件系统中，保留周期为 2 个月

本项目中的后端日志使用 `zap` 库，它由 Uber 开源的一款专为 Go 语言设计的高性能结构化日志库。它通过避免反射、使用强类型设计和零分配内存的 JSON 编码器等技术手段，在保持出色性能的同时提供了结构化日志输出能力。具体的日志配置如下（已表示为 json 格式）：

```
zap:
  prefix: image_hub
  timeFormat: 2006/01/02 - 15:04:05.00000
  level: debug
```

```
encoding: "json"
caller: true
stackTrace: false
# 日志输出到哪里 file | console | both
writer: both
logFile:
  maxSize: 50
  backups: 10
  compress: true
  output:
    - "./log/image_hub.log"
```

八、附录

8.1 项目进度安排

项目的进度安排如下，目前正在稳步推进中：

时间	任务
2025 年 9 月	项目初期分析和调研
2025 年 10 月	完成设计报告
2025 年 11 月	基本完成项目编码
2025 年 12 月	调试和完善项目，并完成测试报告和产品说明书

8.2 备注

本设计报告为项目初期的评估，后续还会根据实际情况进行调整。